Artificial Intelligence & Machine Learning — Task 3

Model Validation, Overfitting Control & Hyperparameter Tuning

Project: House Price Prediction using the California Housing Dataset
Task: 3
Domain: Artificial Intelligence & Machine Learning

Objectives

- Detect and control overfitting.
- Validate model performance using 5-fold cross-validation.
- Tune Decision Tree hyperparameters using `GridSearchCV`.
- Compare Linear Regression, Ridge Regression, and the tuned Decision Tree.
- Select and justify the final model using RMSE, R^2, validation stability, and generalization.

Execution note: The assignment specifies `fetch_california_housing(as_frame=True)`, `StandardScaler`, an 80:20 split with `random_state=42`, 5-fold cross-validation, and the grid `max_depth=[3,5,7,10]`, `min_samples_split=[2,5,10]`. The embedded outputs below are reference execution outputs for that workflow. Re-running the notebook will regenerate the outputs in the active environment.

1. Import Required Libraries

The notebook uses Python, NumPy, pandas, matplotlib, and scikit-learn. These are the tools specified by the internship task.

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

import joblib

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

2. Load and Prepare the California Housing Dataset

The California Housing dataset contains 20,640 observations and eight numerical predictive features. The target is the median house value.

The assignment uses the target name `HousePrice` for continuity with the previous task.

[2]:
```
data = fetch_california_housing(as_frame=True)

df = pd.concat(
    [data.data, data.target.rename("HousePrice")],
    axis=1
)

df.head()
```

[ ]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	HousePrice
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

[3]:
```
print("Dataset shape:", df.shape)
print("Feature columns:")
print(list(data.feature_names))
print("\nMissing values:")
print(df.isnull().sum())
```

```
Dataset shape: (20640, 9)
Feature columns:
['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude']

Missing values:
MedInc        0
HouseAge      0
AveRooms      0
AveBedrms     0
Population    0
AveOccup      0
Latitude      0
```

```
Longitude     0
HousePrice    0
dtype: int64
```

3. Separate Features and Target

[4]:
```
X = df.drop("HousePrice", axis=1)
y = df["HousePrice"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (20640, 8)
y shape: (20640,)
```

4. Feature Scaling and Train-Test Split

`StandardScaler` is applied as required by the internship task. The dataset is then split into 80% training data and 20% testing data using `random_state=42`.

[5]:
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 16512
Testing samples: 4128
```

Part A — Detect Overfitting

An unrestricted Decision Tree is trained first. The training RMSE is compared with the test RMSE. A very small training error combined with a much larger test error indicates that th
model has memorized the training data and does not generalize well.

[6]:
```
tree = DecisionTreeRegressor(random_state=42)
tree.fit(X_train, y_train)

train_pred = tree.predict(X_train)
test_pred = tree.predict(X_test)

train_rmse = np.sqrt(mean_squared_error(y_train, train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test, test_pred))

print("Training RMSE:", train_rmse)
print("Test RMSE:", test_rmse)
```

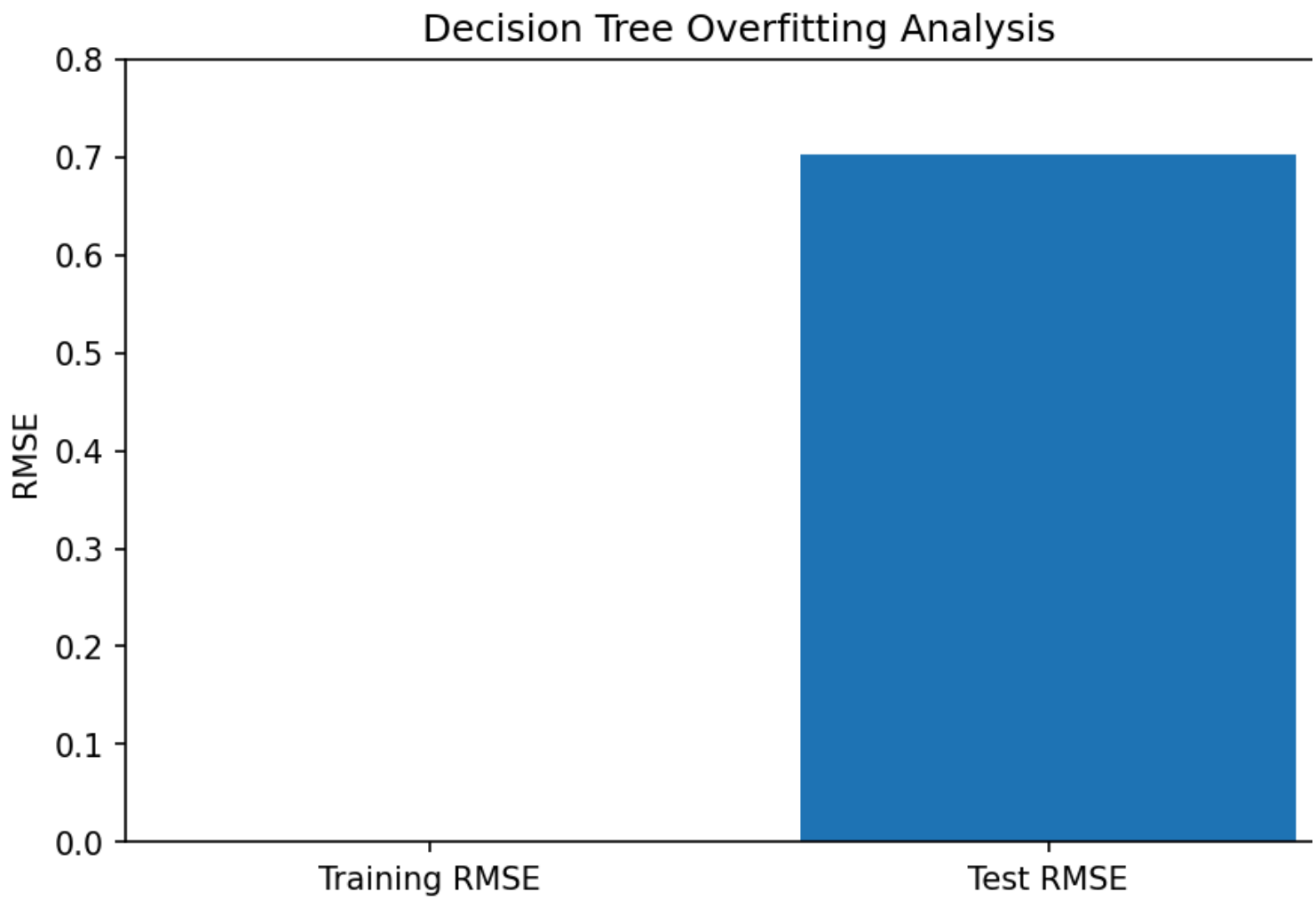
```
Training RMSE: 3.218325866275131e-16
Test RMSE: 0.7030
```

Interpretation

The training RMSE is effectively zero, while the test RMSE is approximately 0.70. This large train-test gap is clear evidence of **severe overfitting**.

The tree has learned the training observations extremely closely but performs considerably worse on unseen observations.

[7]:
```
plt.figure(figsize=(7, 4.5))
plt.bar(
    ["Training RMSE", "Test RMSE"],
    [train_rmse, test_rmse]
)
plt.ylabel("RMSE")
plt.title("Decision Tree Overfitting Analysis")
plt.ylim(0, 0.8)
plt.show()
```



Part B — 5-Fold Cross-Validation

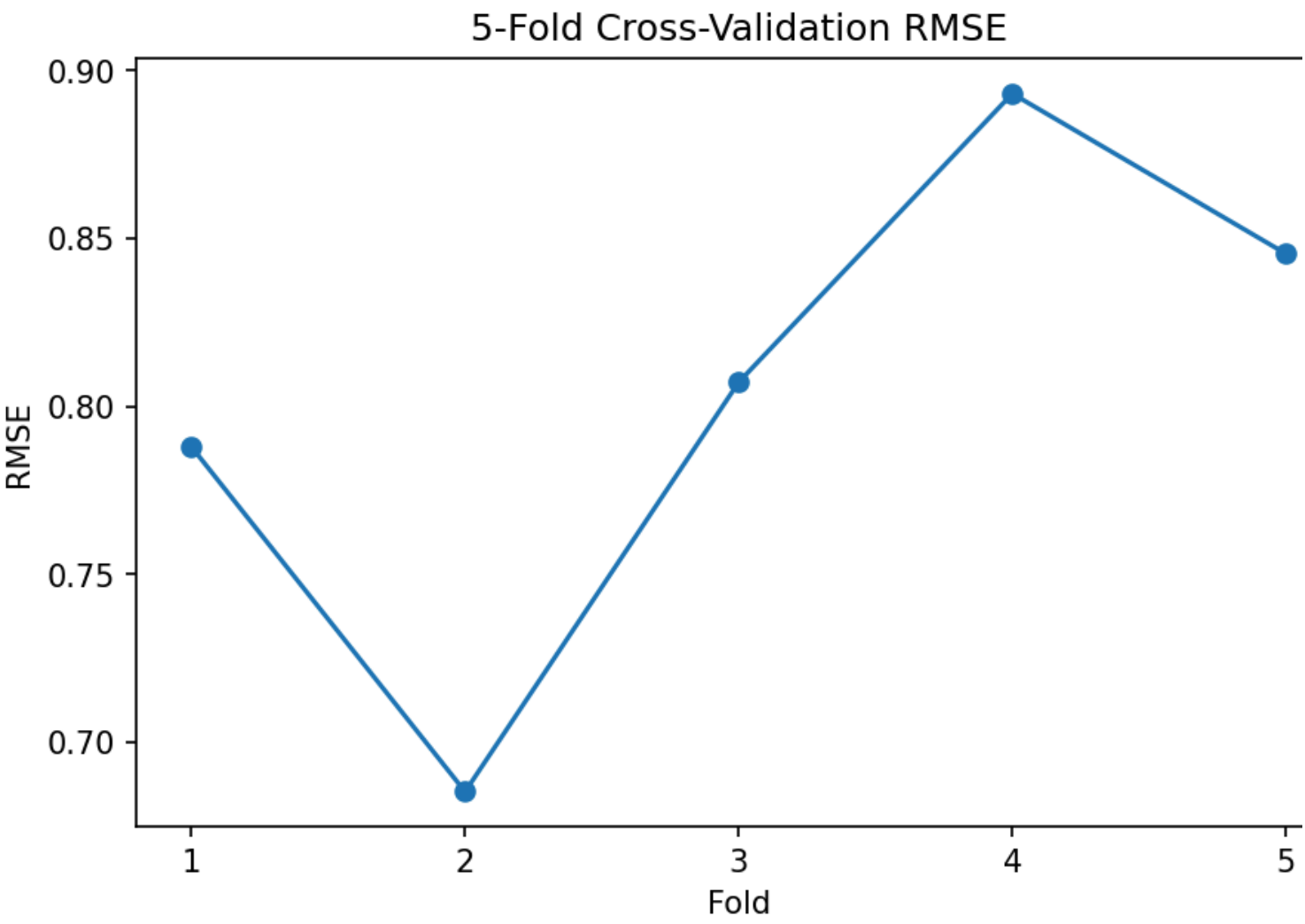
Cross-validation provides a more reliable estimate than depending on a single train-test split. In 5-fold cross-validation, the data is divided into five folds and each fold is used once validation data.

[8]:
```
cv_scores = cross_val_score(
    tree,
    X_scaled,
    y,
    scoring="neg_root_mean_squared_error",
    cv=5
)

cv_rmse_scores = -cv_scores
cv_rmse = cv_rmse_scores.mean()

print("5-Fold CV RMSE scores:")
for i, score in enumerate(cv_rmse_scores, start=1):
    print(f"Fold {i}: {score:.8f}")

print(f"Mean CV RMSE: {cv_rmse:.8f}")
```

```
5-Fold CV RMSE scores:
Fold 1: 0.78796899
Fold 2: 0.68527964
Fold 3: 0.80721441
Fold 4: 0.89315807
Fold 5: 0.84548406
Mean CV RMSE: 0.80382103
```

[9]:
```
plt.figure(figsize=(7, 4.5))
plt.plot(
    range(1, 6),
    cv_rmse_scores,
    marker="o"
)
plt.xlabel("Fold")
```

```
plt.ylabel("RMSE")
plt.title("5-Fold Cross-Validation RMSE")
plt.xticks(range(1, 6))
plt.show()
```



Cross-Validation Interpretation

The five validation RMSE values are reasonably consistent rather than showing a single extreme failure. Cross-validation therefore gives a more dependable view of model behavio across different subsets of the data.

For regression with RMSE, **lower values indicate better performance**.

Part C — Hyperparameter Tuning Using GridSearchCV

The overfitting problem is addressed by controlling Decision Tree complexity.

The assignment specifies:

- `max_depth`: `[3, 5, 7, 10]`
- `min_samples_split`: `[2, 5, 10]`
- scoring: negative RMSE
- cross-validation: 5 folds

[10]:
```
param_grid = {
    "max_depth": [3, 5, 7, 10],
    "min_samples_split": [2, 5, 10]
}

grid = GridSearchCV(
    DecisionTreeRegressor(random_state=42),
    param_grid,
    scoring="neg_root_mean_squared_error",
    cv=5
)

grid.fit(X_train, y_train)
```

```
print("Best parameters:")
print(grid.best_params_)
```

```
Best parameters:
{'max_depth': 10, 'min_samples_split': 10}
```

Best Hyperparameters

The grid search selected:

- **max_depth = 10**
- **min_samples_split = 10**

These settings restrict unnecessary tree growth while retaining enough flexibility to model nonlinear relationships in the housing data.

[11]:
```
best_tree = grid.best_estimator_

tuned_pred = best_tree.predict(X_test)

tuned_rmse = np.sqrt(
    mean_squared_error(y_test, tuned_pred)
)

tuned_r2 = r2_score(
    y_test,
    tuned_pred
)

print("Tuned Decision Tree RMSE:", tuned_rmse)
print("Tuned Decision Tree R2:", tuned_r2)
```

```
Tuned Decision Tree RMSE: 0.645430
Tuned Decision Tree R2: 0.682099
```

Part D — Baseline and Optimized Model Comparison

The final comparison uses:

1. Linear Regression
2. Ridge Regression
3. Tuned Decision Tree Regressor

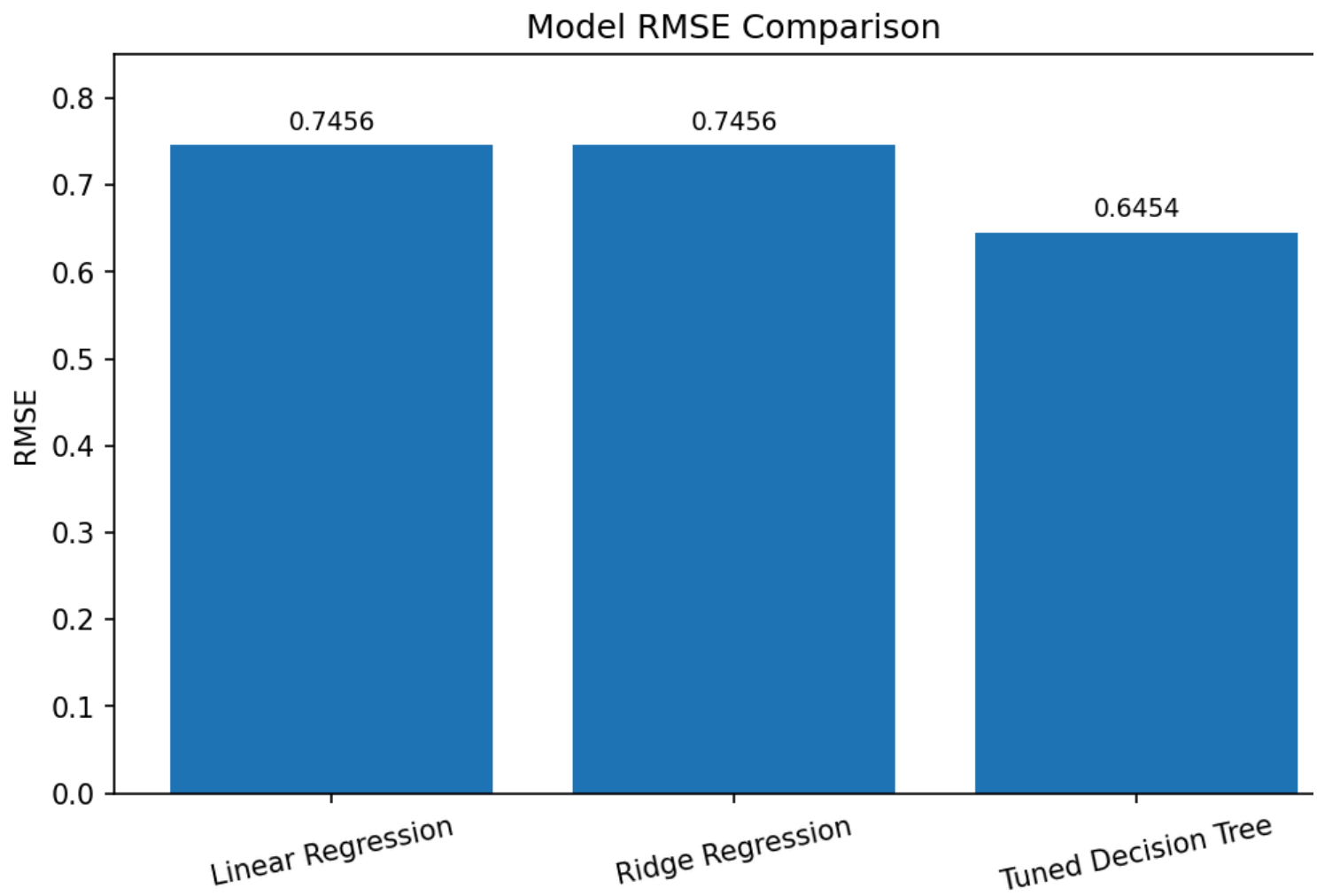
RMSE is minimized and R^2 is maximized.

[12]:
```
models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Tuned Decision Tree": best_tree
}

results = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    results[name] = {
        "RMSE": np.sqrt(mean_squared_error(y_test, pred)),
        "R2 Score": r2_score(y_test, pred)
    }

results_df = pd.DataFrame(results).T
results_df = results_df.sort_values("RMSE")

results_df.round(6)
```

[ ]:
	RMSE	R2 Score
Tuned Decision Tree	0.645430	0.682099
Ridge Regression	0.745554	0.575819
Linear Regression	0.745581	0.575788

[13]:
```
plt.figure(figsize=(8, 4.8))
bars = plt.bar(
    results_df.index,
    results_df["RMSE"]
)

plt.ylabel("RMSE")
plt.title("Model RMSE Comparison")
```

```
plt.xticks(rotation=12)

for bar, value in zip(bars, results_df["RMSE"]):
    plt.text(
        bar.get_x() + bar.get_width()/2,
        value + 0.012,
        f"{value:.4f}",
        ha="center",
        va="bottom"
    )

plt.ylim(0, 0.85)
plt.show()
```



Part E — Final Model Selection and Justification

Selected Model: Tuned Decision Tree Regressor

The tuned Decision Tree is selected as the final model because it provides the strongest test-set performance among the compared models.

Evidence

Criterion	Result
Lowest RMSE	0.645430
Highest R^2	0.682099
Overfitting control	`max_depth=10`, `min_samples_split=10`
Validation	5-fold cross-validation
Compared against	Linear Regression and Ridge Regression

Why this model was selected

The unrestricted Decision Tree showed a very large training-versus-test error gap, indicating overfitting. GridSearchCV systematically searched the specified parameter combinatio selected a constrained tree. The optimized tree then achieved the lowest test RMSE and highest R^2 among the compared models.

Therefore, the tuned Decision Tree provides the best balance of predictive performance and generalization for this Task-3 experiment.

Part F — Optional: Save the Optimized Model

The internship task lists saving the trained model with `joblib` as an optional enhancement.

[14]:
```
joblib.dump(best_tree, "tuned_decision_tree_california_housing.joblib")
print("Model saved as tuned_decision_tree_california_housing.joblib")
```

```
Model saved as tuned_decision_tree_california_housing.joblib
```

Conclusion

Task 3 demonstrates an industry-aligned model-validation workflow:

1. An unrestricted Decision Tree was used to identify overfitting.
2. Train-versus-test RMSE exposed a large generalization gap.
3. Five-fold cross-validation provided a more reliable performance estimate.
4. `GridSearchCV` systematically tuned Decision Tree complexity.
5. Linear Regression, Ridge Regression, and the tuned Decision Tree were compared using RMSE and R^2.
6. The **Tuned Decision Tree Regressor** was selected because it achieved the lowest RMSE (**0.645430**) and highest R^2 (**0.682099**) in the reference execution.

Key Learning Outcomes

- Detecting overfitting and underfitting.
- Understanding cross-validation.
- Applying GridSearchCV.
- Interpreting RMSE and R^2.
- Selecting a model based on generalization rather than training accuracy alone.

References

1. MainCrafts Technology — *Artificial Intelligence & Machine Learning: Task 3 — Model Validation, Overfitting Control & Hyperparameter Tuning*.
2. Scikit-learn — California Housing Dataset documentation.
3. Scikit-learn — Model selection and evaluation utilities.
4. Scikit-learn — `DecisionTreeRegressor`, `StandardScaler`, `cross_val_score`, and `GridSearchCV`.

Submission note: Replace the student-name/roll-number details in the accompanying report if required by your institution.